

GUÍA DE ESTUDIO · CHEATSHEET DE CLI

Comandos de **Angular** y **Next.js**

Del framework Angular 21 (Angular CLI) y de Next.js (App Router) para el desarrollo frontend.

Desde la instalación hasta generar componentes, servicios, páginas y componentes de React.

Angular CLI · `ng`

Next.js · `create-next-app`

TypeScript

Node.js

Documento generado en el contexto del proyecto Demo-IraCar (Compras-Frontend-Local).
Referencia de estudio / repaso rápido.

Índice

Parte A · Angular CLI

1. Requisitos e instalación de Angular CLI
2. Crear y configurar un proyecto
3. Servir, compilar y desplegar
4. Pruebas, lint y formateo
5. Generadores: crear componentes y más (ng generate)
6. Añadir librerías oficiales (ng add)
7. Mantenimiento y comandos auxiliares
8. Tabla rápida de comandos Angular

Parte B · Next.js

1. Requisitos e instalación
2. Crear un proyecto (create-next-app)
3. Scripts de desarrollo, build y producción
4. Estructura App Router: páginas, layouts y rutas
5. Crear componentes de React (server vs client)
6. Añadir librerías: Tailwind, shadcn/ui y MUI
7. Otros comandos útiles
8. Tabla rápida de comandos Next.js

Diferencia clave: Angular tiene un CLI con generadores (`ng generate` / `ng add`). Next.js solo trae el scaffold de proyecto (`create-next-app`); las páginas y componentes se crean como **archivos a mano** dentro de `app/` o `components/` .

Angular CLI

Comandos para crear, servir, probar y generar código con Angular 21 (standalone).

A.1 Requisitos e instalación

Necesitas **Node.js LTS** (incluye `npm`) y el **Angular CLI**, que se instala globalmente:

```
# 1. Verificar Node y npm
node -v      → v22.x.x
npm -v       → 11.x.x

# 2. Instalar Angular CLI de forma global
npm install -g @angular/cli

# 3. Comprobar la instalación y la versión
ng version
```

Tip: si el comando `ng` no se reconoce, cierra y abre la terminal o reinstala con permisos de administrador.

A.2 Crear y configurar un proyecto

```
# Proyecto nuevo con el nombre indicado (Angular 21 crea standalone + SSR por defecto)
ng new mi-proyecto

# Variante sin preguntas (usa valores por defecto)
ng new mi-proyecto --defaults

# Con opciones controladas
ng new mi-proyecto --standalone --routing --style=scss --skip-tests --skip-git --ssr=false --package-manager=npm
```

Flag	Qué hace
<code>--standalone</code>	Crea componentes sin NgModules (estándar en v21)
<code>--routing</code>	Agrega <code>app.routes.ts</code> y el router
<code>--style=scss</code>	Preprocesador de estilos (<code>scss</code> / <code>css</code> / <code>less</code> / <code>sass</code>)
<code>--ssr=false</code>	Habilita o deshabilita Angular Universal/SSR
<code>--skip-tests</code>	No genera archivos <code>.spec.ts</code>
<code>--skip-git</code>	No inicializa un repositorio git
<code>--defaults</code>	Usa todos los valores por defecto sin preguntar

A.3 Servir, compilar y desplegar

```
# Servidor de desarrollo (http://localhost:4200)
ng serve
```

```
# Abrir navegador automático y cambiar de puerto
ng serve -o --port 4200

# Compilar en desarrollo (rápido)
ng build --configuration development

# Compilar en producción (optimizado)
ng build
ng build --configuration production --output-path dist/mi-app
```

Nota del proyecto: en Demo-IraCar se usa `npm start` (= `ng serve`) y `npm run build` para producción (aplica `environment-prod.ts`).

A.4 Pruebas, lint y formateo

```
# Ejecutar pruebas una sola vez (CI) o en modo watch
ng test --watch=false
ng test --code-coverage

# Lint (reglas de estilo/errores de código)
ng lint

# Formateo con prettier (si está en el proyecto)
npx prettier --write "src/**/*.ts"
```

Demo-IraCar usa Vitest; la prueba se lanza con `npx ng test --watch=false`.

A.5 Generadores: crear componentes y más

El comando maestro es `ng generate` (abreviado `ng g`). Todos aceptan `--skip-tests` y `--flat` (sin subcarpeta).

Componente (lo más usado)

```
# ng g c = ng generate component · crea .ts + .html + .css (y .spec.ts si no --skip-tests)
ng g c productos --standalone --skip-tests

# En una carpeta con ruta, todo en un solo archivo
ng g c features/cobro --standalone --skip-tests --inline-template --inline-style
```

Flag de componente	Qué hace
<code>--standalone</code>	Componente standalone (importable directo en rutas, lazy-load)
<code>--skip-tests</code>	No genera el <code>.spec.ts</code>
<code>-t / --inline-template</code>	HTML dentro del <code>.ts</code>
<code>-s / --inline-style</code>	CSS dentro del <code>.ts</code>
<code>--flat</code>	No crea subcarpeta con el mismo nombre
<code>--prefix=app</code>	Prefijo del selector (defecto <code>app</code>)
<code>--change-detection=OnPush</code>	Detección de cambios OnPush

Servicio, guard, pipe, directiva

```
# Servicio (HTTP, lógica de negocio)
ng g s auth --skip-tests

# Guard (protección de rutas: elige CanActivate + funcional)
ng g g permission --skip-tests

# Pipe (transforma valores en el template) y directiva
ng g p paginate --skip-tests
ng g d has-permission --skip-tests
```

Las guardas funcionales (`CanActivateFn`) son el estándar: la `PermissionGuard` de este proyecto es una `CanActivateFn`.

Interfaz, clase, enum y módulo

```
# Interfaz TypeScript (modelo de datos)
ng g i product          # ng g i = ng generate interface

# Clase / enum
ng g class menu-item --type=model
ng g enum payment-method

# Módulo (solo necesario en proyectos no-standalone)
ng g m shared          # ng g m = ng generate module
```

A.6 Añadir librerías oficiales (ng add)

`ng add` instala el paquete **y** ejecuta su schematic (configura el proyecto automáticamente).

```
# Angular Material (tema, tipografía, animaciones – el "ng install material")
ng add @angular/material

# SSR / prerender, PWA, localización y NgRx (estado)
ng add @angular/ssr
ng add @angular/pwa
ng add @angular/localize
ng add @ngrx/store
```

Recuerda: no es `ng install`. Para Material el comando correcto es `ng add @angular/material`. Los paquetes que **no** traen schematic se instalan con `npm install` (ej. `chart.js`, `xlsx`, `jspdf`).

A.7 Mantenimiento y comandos auxiliares

```
# Actualizar Angular (core + CLI + dependencias compatibles)
ng update @angular/core @angular/cli

# Información, configuración y docs
ng version
ng config
ng doc component
```

```
# Diagnóstico/caché
ng analytics # activar/desactivar métricas
ng cache    # estado de la caché de build
```

A.8 Tabla rápida de comandos Angular

Comando	Qué hace
<code>npm install -g @angular/cli</code>	Instala el CLI globalmente
<code>ng new nombre</code>	Crea un proyecto nuevo
<code>ng serve -o</code>	Servidor de desarrollo en :4200
<code>ng build</code>	Compila para producción
<code>ng g c nombre --standalone --skip-tests</code>	Crea un componente
<code>ng g s / ng g g / ng g p / ng g d</code>	Servicio / guard / pipe / directiva
<code>ng g i / ng g class / ng g enum</code>	Interfaz / clase / enum
<code>ng add @angular/material</code>	Añade Angular Material
<code>ng add @angular/ssr</code>	Añade SSR/prerender
<code>ng test --watch=false</code>	Ejecuta las pruebas una vez
<code>ng lint</code>	Analiza el código
<code>ng update @angular/core @angular/cli</code>	Actualiza Angular

Next.js

Comandos para instalar, crear proyectos y generar páginas/componentes con Next.js (App Router).

B.1 Requisitos e instalación

Necesitas **Node.js** (18.18 o superior) y un gestor: `npm`, `yarn`, `pnpm` o `bun`. Next.js se instala por proyecto, no global.

```
# En un proyecto vacío existente (con package.json):
npm install next@latest react@latest react-dom@latest
```

B.2 Crear un proyecto (create-next-app)

```
# Crea la carpeta, pregunta interactivamente (TS, ESLint, Tailwind, src/, App Router, Turbopack, alias)
npx create-next-app@latest mi-app

# Con npm (equivalente):
npm create next-app@latest mi-app

# Todo con flags (sin preguntas)
npx create-next-app@latest mi-app --typescript --app --tailwind --eslint --no-src-dir --import-alias "@/*" --use-npm
```

Flag	Qué hace
<code>--typescript / -ts</code>	Proyecto en TypeScript (+ <code>--js</code> para JavaScript)
<code>--app</code>	Usa App Router (recomendado); sin él usa Pages Router
<code>--tailwind</code>	Configura Tailwind CSS (+ <code>--no-tailwind</code> para quitarlo)
<code>--eslint</code>	Configura ESLint (+ <code>--no-eslint</code>)
<code>--src-dir / --no-src-dir</code>	Crea el código dentro de <code>src/</code> o en la raíz
<code>--import-alias "@/*"</code>	Alias de imports absolutos
<code>--use-npm</code>	Fuerza el gestor (<code>--use-pnpm</code> , <code>--use-yarn</code> , <code>--use-bun</code>)
<code>--turbopack</code>	Activa Turbopack como bundler
<code>--skip-install</code>	No instala dependencias (solo scaffolding)
<code>--empty</code>	Proyecto mínimo sin ejemplos

Nota: a diferencia de `ng new`, `create-next-app` es el **único** scaffolder oficial. Los componentes y páginas siguientes se crean **a mano como archivos**.

B.3 Scripts de desarrollo, build y producción

```
# Servidor de desarrollo (http://localhost:3000, hot reload)
npm run dev
```

```
# Compilar producción a la carpeta .next/
npm run build

# Servir el build de producción (tras npm run build)
npm run start

# Lint y versiones
npm run lint
npx next info
```

Script	Equivalente
<code>npm run dev</code>	<code>next dev</code>
<code>npm run build</code>	<code>next build</code>
<code>npm run start</code>	<code>next start</code>

B.4 Estructura App Router: páginas, layouts y rutas

En Next.js **los archivos definen las rutas**. Un archivo con nombre especial dentro de `app/` crea parte de una página:

Archivo	Qué crea
<code>app/page.tsx</code>	La ruta raíz <code>/</code>
<code>app/layout.tsx</code>	Layout raíz (HTML, fuentes, navegación)
<code>app/productos/page.tsx</code>	Ruta <code>/productos</code>
<code>app/productos/[id]/page.tsx</code>	Ruta dinámica <code>/productos/1</code> , <code>/productos/2...</code> (params <code>{ id }</code>)
<code>app/api/hello/route.ts</code>	API Route <code>/api/hello</code> (export <code>async function GET()</code>)
<code>app/loading.tsx</code>	UI de carga (Suspense automático)
<code>app/error.tsx</code>	UI de error (recibe <code>error</code> y <code>reset</code>)
<code>app/not-found.tsx</code>	UI 404
<code>app/template.tsx</code>	Capa que se remonta en cada navegación

```
# Crear una página nueva = crear el archivo con su carpeta
mkdir app/productos
# → luego escribir en app/productos/page.tsx:

export default function ProductosPage() {
  return <h1>Lista de productos</h1>
}
```

B.5 Crear componentes de React (server vs client)

No hay `ng g component` en Next.js: creas el archivo. Conviene una carpeta `components/` (o `src/components/`):

```
# Componente Server (por defecto, sin interacción):
components/Button.tsx →

export default function Button({ children }) {
```

```

    return <button>{children}</button>
  }

# Componente CLIENT (con useState/onClick): basta declarar "use client" al inicio

"use client"
import { useState } from "react"

export default function Contador() {
  const [n, setN] = useState(0)
  return <button onClick={() => setN(n + 1)}>{n}</button>
}

```

Regla clave: sin `"use client"`, los componentes son Server Components (sin hooks de estado). Añade la directiva solo cuando necesites interactividad.

B.6 Añadir librerías: Tailwind, shadcn/ui y MUI

```

# Tailwind CSS ya viene con create-next-app; en un proyecto sin él:
npm install -D tailwindcss @tailwindcss/postcss
npx tailwindcss init -p

# shadcn/ui (componentes reutilizables sobre Tailwind - tipo "ng add material" de React)
npx shadcn@latest init
npx shadcn@latest add button
npx shadcn@latest add card dialog -y

# MUI (Material Design para React)
npm install @mui/material @emotion/react @emotion/styled

```

B.7 Otros comandos útiles

```

# Telemetría, entorno y análisis del bundle
npx next telemetry disable
npx next info
npx next build # en segundo plano también analiza el bundle

# Despliegue estático (exporta la app a out/)
# configurar output: 'export' en next.config.ts y luego:
npm run build

```

Deploy: Vercel (creador de Next.js) basta con `vercel` + `vercel --prod`; otras opciones: Netlify, Railway, Docker + `next start`.

B.8 Tabla rápida de comandos Next.js

Comando	Qué hace
<code>npx create-next-app@latest mi-app</code>	Crea un proyecto Next.js
<code>npm install next react react-dom</code>	Instala Next en un proyecto existente

<code>npm run dev</code>	Servidor de desarrollo en :3000
<code>npm run build</code>	Compila para producción (.next/)
<code>npm run start</code>	Sirve el build de producción
<code>npm run lint</code>	Analiza el código
<code>archivo app/.../page.tsx</code>	“Genera” una página (la ruta = la carpeta)
<code>archivo components/X.tsx</code>	Crea un componente (React Server/Client)
<code>npx shadcn@latest add button</code>	Añade un componente shadcn/ui
<code>npx next info</code>	Muestra info del entorno

Guía de estudio generada para el proyecto Demo-IraCar (Compras-Frontend-Local). Referencia rápida de Angular CLI y Next.js · reemplaza **TU-APP** por tu dominio al desplegar.